Participatory Texts for Millions: A Paragraph-Centric Decidim Module Born from Production Outages

Abstract

Participatory texts, meaning long normative documents that citizens comment on paragraph by paragraph, are the oldest continuously used digital participation method in Brazil. At federal scale, however, they exceeded what the hosting platform could serve: texts of a few thousand paragraphs took up to a minute to load, and a national consultation caused outages. This article presents decidim-participatory_texts, a standalone open-source engine for Decidim 0.32.x that rebuilds the method around a lean Paragraph model. A recursive parser converts pasted Word and Google Docs content into structured paragraphs, bulk insertion persists up to 2,000 paragraphs in batched inserts, and paginated reading with a comment counter keeps rendering cheap. An integrated feedback workflow with four explicit states links citizen comments to government responses and is exported as open data. In load tests, page load for 2,000 paragraphs fell from 50 s to 1.6 s. The module is published under AGPL-3.0-or-later and is intended as a community-ready alternative to the vote-centric participatory texts functionality in Decidim core.

1 Introduction

A participatory text is a long normative document, such as a bill, a multi-year plan or a regulatory decree, published online so that citizens can read it and comment on each paragraph individually. The unit of participation is the paragraph, not the document or the discussion thread, because the paragraph is the unit in which law and policy are written, amended and negotiated.

In Brazil this method has been in continuous use since 2009, longer than most participation platforms have existed. It has survived three platform generations: a WordPress plugin for the Marco Civil da Internet consultation, the Participa.br platform, and today Brasil Participativo [1], the federal government's multi-tenant Decidim [2] deployment, operated by the Presidency of the Republic and developed by the Lab Livre team at the University of Brasília. The platform currently serves about two million registered users.

This article describes decidim-participatory_texts, the module that now implements the method on Brasil Participativo, and makes three contributions. First, it documents the production failures of 2023–2024 that motivated a rebuild from scratch instead of an incremental fix (Section 2). Second, it presents the resulting architecture in engineering detail, including the recursive paste parser and the bulk persistence path, both based on real documents from federal practice (Section 3). Third, it reports the measured performance gains (Section 4) and compares the module with the participatory-texts functionality shipped in Decidim core (Section 7).

All software described here is open source [3], and every design decision narrated below is visible in the source code comments together with its motivating requirement.

The deployment context shapes several of those decisions. Brasil Participativo is a multi-tenant SaaS: a single Rails application serves many organizations, each isolated in its own PostgreSQL schema, with federated authentication through the national GovBR login. A participatory-texts component must therefore be strictly self-contained, in its tables, its permissions and its exports, because it shares a database with every other organization's

components. The module's refusal to touch core classes and its scoped data model (feedback rows carry a foreign key to the paragraph and never to component-global state) follow directly from that constraint.

2 Statement of need

The requirements of the method are well understood because the same workflow has been operated for fifteen years. Participants read long texts and comment on each paragraph. Public managers, who are not web editors, author in Word or Google Docs and expect the platform to ingest what they paste. The platform must also stay up when a national consultation brings a crowd.

Operation at federal scale between 2023 and 2024 showed that the incumbent implementation failed all three needs (Table 1). Texts with thousands of paragraphs took seconds, sometimes a minute, to load. Upload accepted .docx only, lost formatting without warning, could not represent tables, and the editing page was itself slow. When the 2023 Pluriannual Plan (PPA) consultation brought 1.4 million participants in a single cycle, the platform suffered its first outages.

Table 1. Production findings that motivated the rebuild.

#	Need	Production finding (2023–2024)
1	Read and comment per paragraph	Texts with thousands of paragraphs took seconds to a minute to load
2	Author and edit with ease	.docx-only upload; formatting lost; no tables; slow editing
3	Survive crowds	~2 million users; outages under the 2023 PPA peak

Profiling identified a single dominant cause: rendering, and not querying. For a 600-paragraph text, 96.7% of total page time was spent in the view layer. The incumbent design stored the document as one monolithic record whose children were loaded and rendered one by one, and each paragraph carried the full generic Decidim component machinery (votes, followers, category associations, proposal states) although a participatory text needs none of it. Citizens were commenting on paragraphs of law, not on proposals. The fix was therefore to stop using the generic path instead of optimizing it: the component was rebuilt around a paragraph model that carries only what the method requires.

3 Architecture and features

The module is a self-contained Decidim engine, with no fork of decidim-core and no overrides of other components. It is organized around five design decisions: a lean paragraph model, a recursive paste parser, bulk persistence, a bounded reading interface and a structured feedback workflow.

3.1 A paragraph model that stays lean

The base unit is a Paragraph record with a translatable title and body (JSONB, following Decidim's localization conventions), a typed level, a list position, visibility and interactivity flags, and a publication timestamp. The levels distinguish the structural roles a normative document has: section and sub-section (headings), article (commentable body text), table, image, and a special comments-section placeholder. Comments attach directly to a

paragraph through Decidim's standard commentable concern, so threads, moderation and notifications behave exactly as administrators already know.

The model deliberately includes no voting, following or categorization machinery, so each paragraph renders as a lightweight record and the reading interface stays cheap (Section 3.4). Paragraphs support draft and published states individually and are managed with acts_as_list, so an administrator can reorder, merge or remove paragraphs without invalidating the document. These operations would be impossible if the text were stored as a single document blob.

3.2 From paste to structure: a recursive parser

Managers author in Word or Google Docs, and the paste-driven authoring interface meets them there. The pasted content arrives as editor-generated HTML, and a recursive parser in the ingestion form turns it into paragraph candidates.

Three quirks of real-world input drive the design, and each is documented in the code. First, whitespace must be normalized before parsing: otherwise every line break and indentation in the pasted HTML becomes a stray text node, and the splitter fragments the document into noise. Second, Google Docs wraps the entire document in a <b> element with no semantic weight, which the editor converts to <p>, so all paragraphs end up nested inside a text modifier. The parser detects text modifiers used as divs and recurses through them; without this rule an entire document collapses into a single paragraph. Third, lists are treated as units and not as one paragraph per item: a list is a single indivisible block, which matches how normative text is numbered and referenced.

Elements are then classified by tag and content: h1 becomes a section, h2 and h3 become sub-sections, table becomes its own interactive level, image-only elements become image paragraphs titled by their alt text, and everything else is an article. Empty elements, including the whitespace-only residue of normalization, are dropped. Tables are the exception, since they are never empty by definition.

3.3 Bulk persistence for legislative-size documents

The size target for the module is the LDO, Brazil's budget guidelines law, whose 2027 edition has almost 1,800 numbered devices. Inserting paragraphs row by row cost seven queries per paragraph and pushed a full paste past fifteen seconds, close to the request timeout. The ingestion command instead builds all rows in memory and persists them with insert_all! in batches of 250 inside a single transaction. This costs one database round-trip per batch, with a hard cap of 2,000 paragraphs per paste, and positions are pre-computed as an offset from the document's current tail, so appending never reorders what is already published. Validation shifts accordingly: structural problems are caught by the parser before any write, and a failed batch rolls the whole paste back.

3.4 A reading interface bounded by construction

The public index paginates ten paragraphs per page. Pagination is more than a rendering convenience: together with a denormalized comments_count counter column, it bounds per-page work regardless of document size, a property the incumbent design lacked. Each paragraph receives a stable ordinal computed from its list position, so citizens and administrators can refer to "article 12" unambiguously. Full-text search strips HTML tags in the database with a regular expression before matching, which keeps search semantics on

the text and not on the markup. An authenticated preview mode lets administrators page through drafts exactly as citizens will see them after publication.

3.5 Closing the loop: structured feedback on every comment

The module closes the loop between citizen comment and government response. It attaches a CommentFeedback record to each citizen comment on an interactive paragraph. The record is authored by an administrator and has a status drawn from four explicit states (awaiting_review, incorporated, partially_incorporated, not_incorporated) and a free-text explanation that is mandatory whenever a decision has been made. The constraint is enforced at the form, model and database levels, so an official response (*devolutiva*) without an explanation cannot exist. Comment threads on paragraphs are additionally exported through Decidim's export framework with include_in_open_data: true, so the trace from citizen input to government response remains auditable outside the platform.

Commenting itself is governed per participatory-process step (disabled by default, with a configurable maximum length), and only interactive, published paragraphs accept new comments. Sections and tables of contents never do.

3.6 Administrative operations

Administration is built on Rails and Turbo Streams: moving, merging, destroying, hiding or toggling interactivity of paragraphs updates the editor in place without a full reload. Merging combines adjacent paragraphs while preserving comment associations, and the component refuses to be removed from a space while paragraphs exist, which protects published consultations from accidental destruction.

3.7 Explicit non-goals

Three capabilities were deliberately left out, and the omissions are part of the design.

- **Voting.** Paragraph-level supports or endorsements are meaningless in normative texts whose devices are mutually dependent. The module offers comment plus structured feedback instead.
- **Rich-text citizen annotations.** Citizens comment in plain text, with threading provided by Decidim's commentable concern. Inline markup of the document body is an administrative authoring capability and not a participatory one.
- **Real-time collaborative editing.** The authoring model is intentionally paste-and-review, which matches how legislative drafting offices work: the document of record lives in the drafting office's tools, and the platform ingests snapshots of it.

Each omission removes machinery that the production profile showed dominating rendering cost.

4 Evaluation

The rebuild was validated with load tests on production-scale documents (Table 2). Page load for a 2,000-paragraph text fell from 50 s to 1.6 s, a 31-fold reduction, and inline rendering of 600 paragraphs fell from 34 s to 1.3 s, a 26-fold reduction.

Table 2. Rendering performance before and after the rebuild.

Scenario	Before	After
Page load, 2,000 paragraphs	50 s	1.6 s
Inline render, 600 paragraphs	34 s	1.3 s

The gains follow from three design decisions. Lean records reduce per-paragraph render cost by roughly an order of magnitude, and the 96.7% of page time that rendering consumed shrinks proportionally. Bounded pagination caps per-page work at ten paragraphs regardless of document size, so load time no longer scales with the length of the law. Bulk insertion addresses the write path, which Table 2 does not measure: it removes the N-round-trip pathology and reduces paste persistence from a timeout risk to a sub-second operation. As a result, the component handles production-scale documents within interactive response times.

5 Limitations

The 2,000-paragraph paste cap is a product decision and not a technical ceiling. Larger documents can be pasted in successive appends, but a single legislative text beyond that size has not appeared in federal practice, and the LDO, the largest document class served, is comfortably under it. Search operates on the current locale’s text with tags stripped at query time, which is sufficient for consultation-scale corpora but is not a substitute for a proper full-text index. Comment feedback is administrator-authored only; peer-to-peer discussion remains in the comment threads proper. Finally, the module targets Decidim 0.32.x specifically: it makes no attempt at multi-version compatibility, and each Decidim upgrade is expected to touch the module deliberately, as the 0.32 upgrade commit already did.

6 Use case: from the 2023 PPA to the 2027 LDO

Two cases illustrate the operational challenges that motivated the module's redesign. The first is the Plano Clima (National Climate Plan), which involved public participation across multiple lengthy policy documents, engaging several thousand participants. The second is the public consultation on Brazil's National Driver's License (CNH), which attracted over 40,000 participants submitting comments. Both experiences exposed scalability limitations in the previous implementation, leading to performance degradation and service outages under intensive use. These challenges reinforced the need for an architecture capable of handling both complex documents and large volumes of citizen contributions. The redesigned module addresses these limitations through lightweight paragraph records, batched document ingestion, paginated reading, and a structured feedback workflow. Although its deployment is still at an early stage, initial uses have already shown satisfactory results, supported by significant improvements in performance testing. Public managers can now more efficiently prepare, publish, and manage consultations, while citizens can navigate lengthy documents and submit paragraph-level comments.

The rebuild also changed who can run a participatory-text consultation. Under the incumbent component, loading a legislative-size text required developer intervention to survive publication day. With paginated reading, batched ingestion and bounded rendering, a public manager can paste, preview, publish and moderate without involving operations. The performance numbers are a proxy for this transfer of capability from the development team to the drafting office.

7 Related work

Decidim core ships participatory-texts functionality [4] in which an uploaded document is split into sections that materialize as proposals, inheriting the full proposal machinery: supports, endorsements and amendment workflows. That design serves consultations whose output is a ranked set of ideas. The Brazilian method differs, because its output is a marked-up document in which the government must state, per paragraph, what was incorporated and what was not. Voting on paragraphs has no meaning in a normative text whose devices are mutually dependent, whereas commenting with a guaranteed answer does. The module therefore omits votes and implements the *devolutiva* workflow that the core path lacks.

Architecturally, the module follows the customization patterns validated across the Brasil Participativo ecosystem [1]: behavior rides on standard engine mechanisms and not on core patches, each feature is documented with its originating requirement, and generalizable features are flagged as upstream candidates. This matters because upstream feature proposals are currently closed on MetaDecidim [5], so well-documented modules serve as the de facto proposal backlog for the Decidim developer community.

Beyond Decidim, few direct precedents exist for structured public commenting. Parliamentary and regulatory commenting systems in other jurisdictions, from notice-and-comment portals to document-centric consultation sites, typically treat the comment as the atomic object, attached to a document or a coarse section. In the design described here, the paragraph is the first-class record, and comments, feedback states, search ordinals and chat references all attach to it. This makes the four-state feedback workflow enumerable and exportable per device, and it lets a multi-channel front end address individual articles by a stable number. To our knowledge, no open-source participation platform ships paragraph-level commenting with a structured, open-data government-response workflow as a standalone, reusable component.

8 Availability

The module is published at https://gitlab.com/lappis-unb/decidimbr [3] under the AGPL-3.0-or-later license. It targets Decidim 0.32.x on Ruby ~> 3.3 and is currently at version 0.0.1, pending its first community release; a permanent DOI will be minted through Zenodo at release time. Issue reports and contributions are welcome on the project tracker.

9 Conclusion

The method has been operated for fifteen years and now serves about two million users, and the generic implementation could not keep up. The rebuild is small, about 1,600 lines of Ruby: a paragraph model that carries only what participatory texts need, a parser that handles the documents governments actually paste, bulk persistence sized for legislative documents, and a feedback workflow that gives every comment an explicit, auditable answer. In load tests, page load for 2,000 paragraphs fell about 31-fold and inline rendering of 600

paragraphs about 26-fold. The method has outlasted three platforms, and the module keeps its basic unit, the paragraph, as the central record.

References

[1] Brasil Participativo: module ecosystem on Decidim 0.32.x. https://gitlab.com/lappis-unb/decidimbr

[2] Decidim: free open-source participatory democracy framework. https://decidim.org

[3] decidim-participatory_texts module source and README (repository decidim-participatory_text). https://gitlab.com/lappis-unb/decidimbr/components-brasil-participativo/decidim-participatory_text

[4] Decidim documentation: participatory texts. https://docs.decidim.org

[5] MetaDecidim: community process and proposals. https://meta.decidim.org